

Министерство науки и высшего образования Российской Федерации

Томский государственный университет
систем управления и радиоэлектроники

С.М. Алфёров
А.К. Лукьянов

ВЫЧИСЛИТЕЛЬНЫЕ СИСТЕМЫ

Методические указания по самостоятельной и индивидуальной
работе для студентов всех форм обучения
и направлений магистратуры

Томск
2024

УДК 681.3
ББК 32.97
А-91

Рецензент:

Лукьянов А.К., доцент кафедры Автоматизированных систем управления
ТУСУР, к.т.н.

Алфёров, Сергей Михайлович

А-91 Вычислительные системы: методические указания по самостоятельной и индивидуальной работе для студентов всех форм обучения и направлений магистратуры/ С.М. Алфёров – Томск: Томск. гос. ун-т систем упр. и радиоэлектроники, 2024. – 17 с.

Методические указания содержат задания к лабораторным работам, краткий теоретический материал для выполнения заданий и варианты.

Одобрено на заседании каф. Автоматизированных систем управления,
протокол №11 от 23.11.2023.

УДК 681.3
ББК 32.97

© Алфёров С.М., 2024
© Томск. гос. ун-т систем упр. и
радиоэлектроники, 2024

Оглавление

1. Лабораторные работы.....	4
1.1 Обработка массивов на языке Ассемблер.....	4
1.2 Операции над числами с плавающей точкой.....	5
1.3 Векторные операции над целыми числами.....	8
1.4 Векторные операции над дробными числами.....	14
2. Список рекомендуемой литературы.....	17
2.1. Основная литература.....	17
2.2. Дополнительная литература.....	17
2.3. Перечень пособий, методических указаний и материалов, используемых в учебном процессе.....	17

1. Лабораторные работы

1.1 Обработка массивов на языке Ассемблер

1. ЦЕЛЬ РАБОТЫ

Познакомиться со способом хранения массива данных в памяти ЭВМ и получением доступа к элементам массива.

2. УКАЗАНИЯ К РАБОТЕ

Если массив объявлен статически, то имя массива уже подразумевает массив ячеек (см. рис.1.1 а), в этом случае доступ к массиву можно получить путем непосредственного обращения (через имя массива) или с помощью команды `lea` (получения адреса переменной). Если массив объявлен динамически, то имя массива подразумевает ячейку, которая содержит адрес массива (рис. 1.1 б), поэтому доступ осуществляется через индексный регистр при помощи команды `mov`. При работе с массивами следует учитывать, что индексация ячеек на языке высокого уровня идет по размеру ячеек (индексы сверху ряда ячеек на рис.1.1), а индексация ячеек на языке Ассемблер идет по байтно, не зависимо от размера ячеек (индексы снизу ряда ячеек на рис.1.1).

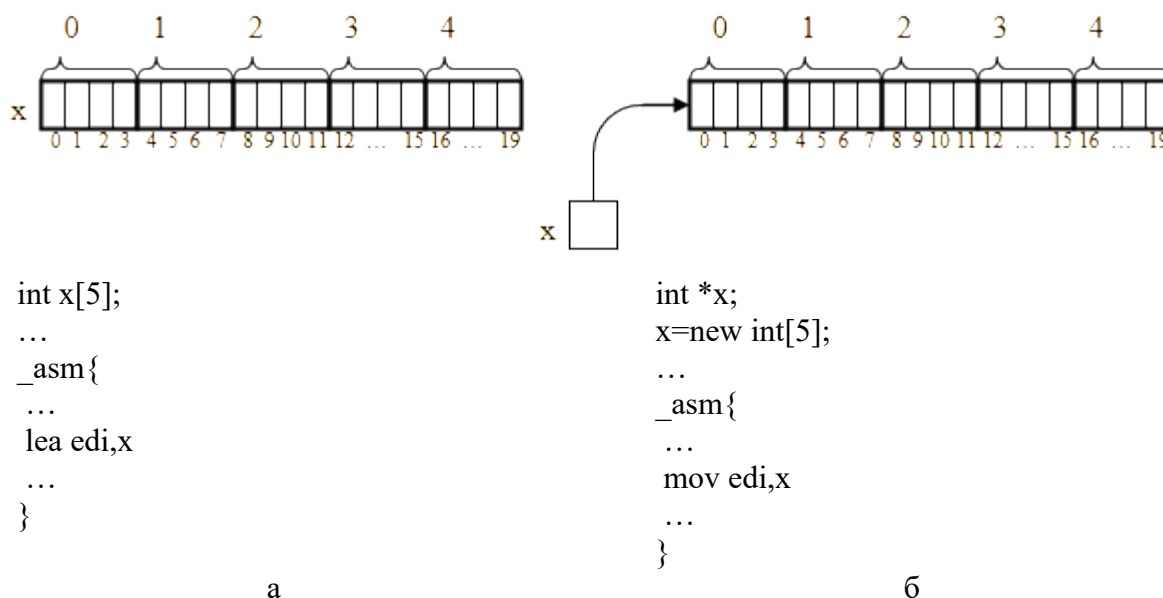


Рис.1.1. Расположение массива в памяти ЭВМ.

Пример сложения всех элементов массива показан в табл. 1.1.

Таблица 1.1. – Пример сложения элементов массива.

Пример на C++	Пример на Ассемблере
<pre> int a[5], b[5], c[5]; ... for (int i=0;i<5; i++){ c[i]=a[i]+b[i]; } </pre>	<pre> int a[5], b[5], c[5]; ... _asm{ lea esi, a lea edi, c lea ebx, b sub ebx, esi mov ecx, 1 @l: </pre>

	<pre> cmp ecx, 5 jae @ex mov al, [esi] //a add al, [ebx+esi] // b mov [edi], al add esi, 4 add edi, 4 inc ecx jmp @l @ex: } </pre>
--	--

3. ЗАДАНИЕ

Обработать одномерный массив из n целых чисел в соответствии с вариантом задания.

1. Вычислить сумму отрицательных элементов.
2. Вычислить сумму положительных элементов.
3. Вычислить сумму элементов массива с нечетными номерами.
4. Найти минимальный элемент массива.
5. Найти максимальный элемент массива.
6. Найти максимальный по модулю элемент массива.
7. Найти минимальный по модулю элемент массива.
8. Посчитать количество элементов, лежащих в диапазоне от А до В.
9. Посчитать количество элементов, которые меньше А.
10. Посчитать количество элементов, которые больше А.

1.2 Операции над числами с плавающей точкой

1. ЦЕЛЬ РАБОТЫ

Ознакомление с системой команд FPU.

2. УКАЗАНИЯ К РАБОТЕ

Модуль обработки чисел с плавающей точкой (FPU) содержит:

- 8 отдельно адресуемых 80-ти битных регистра, организованных в стек (рис. 1.2);
- 3 служебных 16-битных регистра (рис.1.2-1.4): регистр состояния swr (status word register), регистр управления cwr (control word register), регистр тегов twr (tags word register);
- регистр указатель данных dpr (data point register), регистр указатель команд ipr.

	знак 79	порядок	мантисса 0		twr
R0					1 – 0
R1					3 – 2
R2					5 – 4
R3					7 – 6
R4					9 – 8

R5					11 – 10
R6					13 – 12
R7					15 – 14

Рис.1.2 - Структура стека сопроцессора.

Указатель `top` содержит адрес вершины стека `st(0)`, адрес произвольного регистра стека `st(i)` равен `top+i`. При загрузке числа в стек, значение указателя `top` уменьшается, при выгрузке `top` увеличивается.

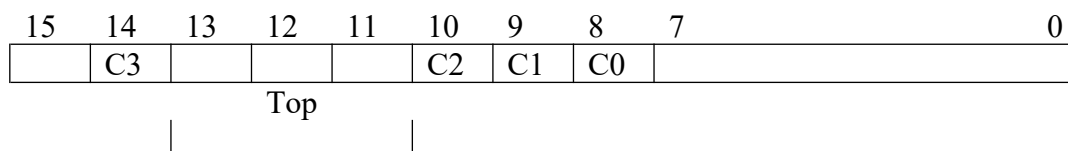


Рис.1.3 - Структура SWR

Регистр swr содержит поля управления точностью (pc) и поля управления округлением (rc).

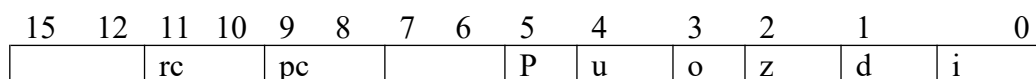


Рис. 1.4 - Структура CWR

Поле `rs` задает длину мантиссы:

1. 00 – 24 бита
2. 10 – 53 бита
3. 11 – 64 бита

Поле `rs` задает способ округления:

1. 00 – $\text{st}(0)$ округляется до ближайшего целого
2. 01 – $\text{st}(0)$ округляется в меньшую сторону
3. 10 – $\text{st}(0)$ округляется в большую сторону
4. 11 – $\text{st}(0)$ отбрасывается дробная часть

Регистр тегов. Поле, соответствующее своему регистру (R0 – R7) содержит признак содержимого регистра:

1. 00 – допустимое ненулевое значение
2. 01 – нулевое значение
3. 10 – одно из специальных значений, кроме нуля
4. 11 – регистр пуст (допускает запись значения)

Команды перемещения данных

Команды для вещественных чисел

Fld <источник> загружает операнд в стек

Fst <приемник>, Fstp <приемник>, записывает значение из стека в приемник. Fstp выталкивает из стека.

Команды для целых чисел

Fild <источник> загружает операнд в стек

Fist <приемник>, Fistsp <приемник>, записывает значение из стека в приемник. Fstrp выталкивает из стека.

$\text{Fych st}(i)$ – поменять значение с вершиной

Загрузка констант

Fldz – 0

Fld1 – 1

Fldpi – π

Fld2t – $\log_2(10)$

Fld2e – $\log_2(e)$

Fldlg2 – $\log_{10}(2)$

Fldln2 – $\ln(2)$

Перед использованием сопроцессора следует выполнить команду Finit, алгоритм которой следующий:

1. cwr = 37h, при этом rc=00
2. В биты с 0-го по 5-й записываются «1», что означает маскирование всех исключений
3. pc=11, максимальная точность
4. swr=0, т.е. исключения отсутствуют и вершиной стека становится R0
5. twr=0ffffh, т.е. все регистры объявляются путыми.
6. dpr = 0, ipr=0

Арифметические команды

Команды сложения и вычитания, их форматы и действия приведены в таблице 1.2.

Таблица 1.2 – Команды сложения и вычитания.

Команда	Действие
Fadd приемник, источник	Приемник = приемник + источник
Faddp приемник, ST	(после сложения, из стека выталкивается значение)
Fadd источник	St(0) = st(0) + источник
Fadd	St(0) = st(0) + st(1)
Fsub приемник, источник	Приемник = приемник - источник
Fsubp приемник, ST	Приемник = приемник – ST(0) выталкивается из стека значение в st(i-1)
Fsub источник	St(0) = st(0) + источник
Fsub	St(0) = st(0) + st(1)
Fsubr st(i), st	St(0)=st(0)-st(i)
Fsubrp st(i), st	St(0)=st(0)-st(i); выталкивание из стека и сохранение в st(i-1)

Аналогично командам сложения и вычитания, команды умножения и деления: fmul, fdiv. Команды тригонометрии: fsin, fcos, fsincos, fptan, fpatan. При необходимости с ними можно познакомиться в [3].

3. ЗАДАНИЕ

Вычислить математическое выражение заданное вариантом, используя команды FPU (см. таблицу 1.2).

Таблица 1.2 – Варианты заданий

1) $Y = A \cdot B + \frac{C^2}{D}$	10) $Y = \frac{C^2}{A} + B + \frac{B-C}{C} D$
------------------------------------	---

2) $Y = \frac{A \cdot B + C}{C} - \frac{C}{D}$	11) $Y = \frac{C^2}{A} + B + \frac{B-D}{D}C$
3) $Y = \frac{A+B}{D} + CD$	12) $Y = AB^2 + \frac{(A-B)(C-D)}{B}$
4) $Y = C - \frac{A}{B} + B^2 - A + D$	13) $Y = \frac{AB}{C} + (A-B)(D-C)$
5) $Y = \frac{AC}{B} + \frac{D^2}{C} - A$	14) $Y = BC + A + \frac{B-C}{B} - D$
6) $Y = \frac{C-A}{B} + AB + D$	15) $Y = C + A^3 + D - B + BA$
7) $Y = AB + C + \frac{D}{B} + A$	16) $Y = (B-C)(B+C) + \frac{BA}{C}$
8) $Y = A^2 - \frac{B}{D} + CB + A$	17) $Y = A^3 + B + \frac{C^2(B-C)}{B}$
9) $Y = CB + A + \frac{DA}{B}$	

1.3 Векторные операции над целыми числами

1. ЦЕЛЬ РАБОТЫ

Ознакомление с системой команд MMX.

2. УКАЗАНИЯ К РАБОТЕ

MMX (MultiMedia eXtensions) предназначена для векторных операций над целыми числами (массивами целых чисел): над массивами из 8-ми байтов, над массивами из 4-х слов, над парами двойных слов.

Типы арифметик:

1) Циклическая, например, при сложении двух байтов 200+100 получим результат 44

2) с насыщением,

а. беззнаковая, при сложении двух байтов 200+100 получим результат 255

б. знаковая, при сложении двух байтов 100+100 получим результат 127

Структура команды:

Большинство команд имеют структуру:

<приставка><«корень»><суффикс>

Приставка означает, к какой системе команд относится команда. Корень означает, какое действие выполняет команда. Суффикс означает тип (размер) операндов или тип арифметики.

Регистры

В MMX используются 64-битные регистры mm0, mm1, ..., mm7. Эти регистры являются частью регистров сопроцессора (FPU), 80-ти битные R0, R1, ..., R7. Каждый 64-

битный регистр (четверное слово) можно представить как массив из: 8-ми байтов, 4-х слов или 2 двойных слов.

Команды

приставка:

p – MMX-команда, f – FPU-команда.

суффикс:

b – операция над массивом байтов.

w - операция над массивом слов (2-х байтных).

d - операция над массивом двойных слов (4-х байтных).

При наличии двух вышеуказанных букв, первая соответствует входному, вторая выходному операнду.

При наличии следующих суффиксов используется арифметика с насыщением, при их отсутствии используется циклическая арифметика:

s или ss - операция над массивом знаковых чисел (с насыщением).

us - операция над массивом беззнаковых чисел (с насыщением).

При совместном использовании FPU и MMX-расширения, последней MMX-командой должна быть команда emms.

Команды переноса (копирования) данных

movd/movq

Формат команд

Команда операнд1, операнд2

Ограничения для этих команд приведены в таблице 1.3.

Таблица 1.3 – Ограничения для MMX-команд переноса данных.

Операнд1	Операнд2	MMX-регистр	Ячейка памяти
MMX-регистр		Да	Да
Ячейка памяти		Да	Нет

Типы операндов приведены в таблице 1.4.

Таблица 1.4 – Типы операндов для MMX-команд переноса данных.

Команда	Длина операндов
Movd	Двойное слово
Movq	Четверное слово

Команды сложения и вычитания

Формат команд

Команда операнд1, операнд2

Ограничения для арифметических команд отличаются от ограничений для команд переноса (см. таблицу 1.5).

Таблица 1.5 – Ограничения для MMX-арифметических команд.

Операнд1	Операнд2	MMX-регистр	Ячейка памяти
MMX-регистр		Да	Да
Ячейка памяти		Нет	Нет

Команды сложения и вычитания по типам операндов и типам арифметик приведены в таблице 1.6.

Таблица 1.6 – Команды для различных типов арифметики и типов данных.

Арифметика	Длина операндов		
	байт	слово	Двойное слово
Циклическая	paddb / psubb	Paddw / psubw	Paddd / psubd
Знаковая с насыщением	Paddsb / psubsb	Paddsw / psubsw	Paddsd / psubsd
Беззнаковая с насыщением	Paddusb / psubusb	Paddusw / psubusw	Paddusd / psubusd

В таблице 1.7 приведены подпрограммы для вычисления суммы соответствующих элементов двух массивов (однобайтовых) в двух вариантах: с использованием системы команд MMX и с использованием скалярной системы команд.

Таблица 1.7 – Два варианта сложения двух массивов.

<pre> procedure TForm1.add_mmx; asm pusha push ebx lea esi,self.a lea edi,self.d lea ebx,self.b sub ebx,esi mov ecx,1 @l: cmp ecx,n jae @ex movq mm0,qword ptr [esi] //a paddb mm0,qword ptr [ebx+esi] // b movq qword ptr [edi],mm0 add esi,8 add edi,8 add ecx,8 jmp @l @ex: emms pop ebx popa end; </pre>	<pre> procedure TForm1.add_x86; asm pusha push ebx lea esi,self.a lea edi,self.c lea ebx,self.b sub ebx,esi mov ecx,1 @l: cmp ecx,n jae @ex mov al,[esi] //a add al,[ebx+esi] // b mov [edi],al inc esi inc edi inc ecx jmp @l @ex: pop ebx popa end; </pre>
---	---

Ниже приведенная программа показывает использование подпрограмм.

```

procedure TForm1.Button1Click(Sender: TObject);
var i:integer;
    t1,t2,t3:tdatetime;

```

```

    ok:boolean;
begin
    t1:=time;
    //for i:=1 to n do c[i]:=a[i]+b[i];
    add_x86;
    t2:=time; add_mmx; t3:=time;

    i:=1; ok:=true;
    while (i<=n) and ok do begin
        ok:=(c[i]=d[i]); inc(i);
    end;
    if ok then label3.Caption:='ok' else label3.Caption:='not ok';
    t1:=t2-t1; t2:=t3-t2;
    // label4.Caption:='pas:'+inttostr(round(t1*24*3600*1000));
    label4.Caption:='x86:'+inttostr(round(t1*24*3600*1000));
    label5.Caption:='mmx:'+inttostr(round(t2*24*3600*1000));
end;

```

Команды упаковки и распаковки

Формат команд

Команда операнд1, операнд2

Ограничения для команд упаковки и распаковки приведены в таблице 1.8.

Таблица 1.8 – Ограничения MMX-команд упаковки и распаковки.

Операнд1	Операнд2	MMX-регистр	Ячейка памяти
MMX-регистр		Да	Да
Ячейка памяти		Нет	Нет

Упаковка – уменьшение разрядности, Команды упаковки по типам операндов приведены в таблице 1.9.

Таблица 1.9 - Команды упаковки по типам операндов.

Наличие знака	Слова в байты	Двойные слова в слова
Знаковые	Packsswb	Packssdw
Беззнаковые	Packuswb	—

Формат команды Packsswb приведен ниже, действие команд см. на рисунке 1.5.

Packsswb MMX-регистр, операнд2

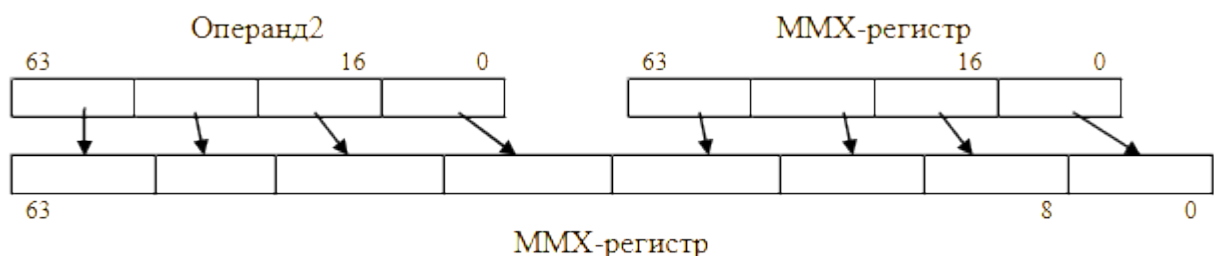


Рисунок 1.5 – Действие команды packsswb.

Формат команды packssdw приведен ниже, действие команд см. на рисунке 1.6.
packssdw MMX-регистр, операнд2

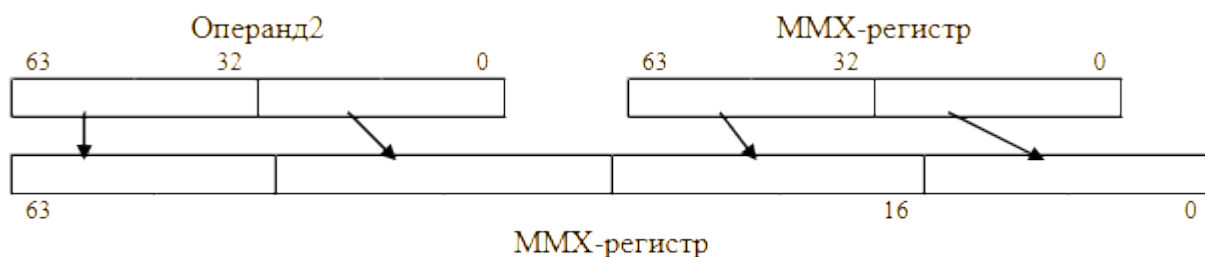


Рисунок 1.6 – Действие команды packssdw.

Формат команды packuswb приведен ниже, действие аналогично packsswb, только в данном случае упаковываются беззнаковые числа.

packuswb MMX-регистр, операнд2

Команды распаковки:

punprckhbw, punprckhwd, punprckhdq

Выполняют попарное объединение элементов из старших частей операндов в первый операнд, элементы из 1-го операнда помещаются в младшую часть пары, из 2-го операнда в старшую часть пары.

Формат команды Punprckhwd приведен ниже, действие показано на рисунке 1.7.

Punprckhwd MMX-регистр, операнд2

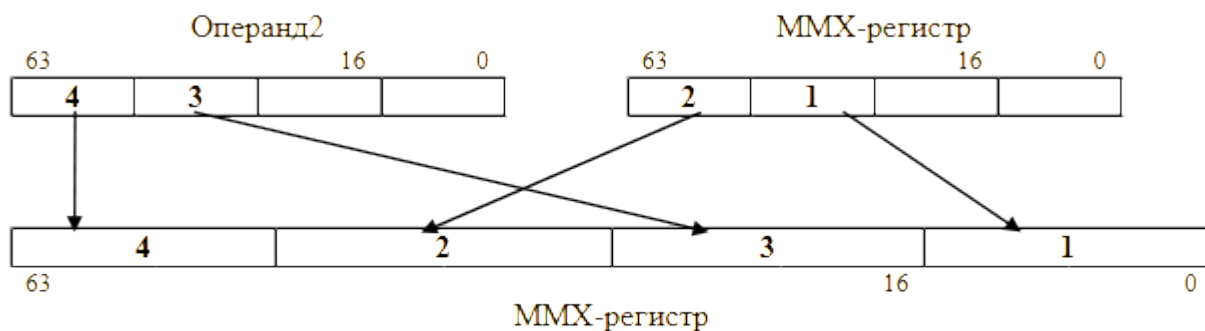


Рисунок 1.7 – Действие команды punprckhwd.

Команды распаковки:

punprcklbw, punprcklwd, punprckldq

Выполняют попарное объединение элементов из младших частей операндов в первый операнд, элементы из 1-го операнды помещаются в младшую часть пары, из 2-го операнда в старшую часть пары.

Формат команды punprcklwd приведен ниже, действие показано на рисунке 1.8.

Punprckhwd MMX-регистр, операнд2

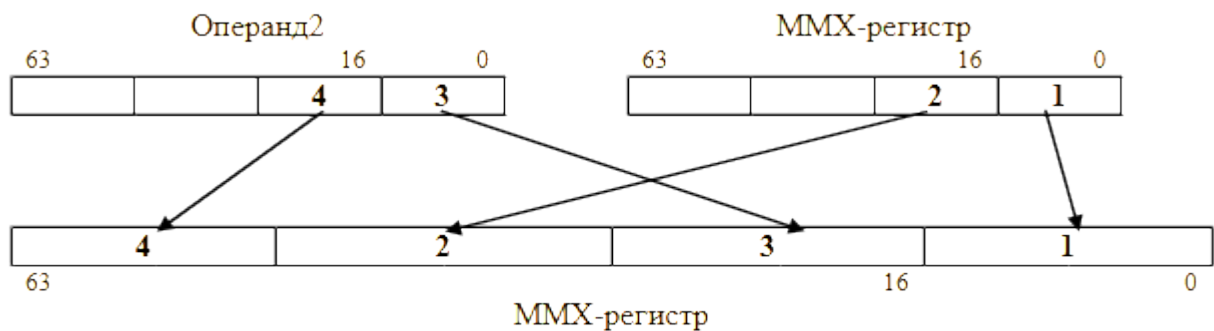


Рисунок 1.8 – Действие команды punpcklwd.

Команды умножения

Pmaddwd

Попарное умножение слов со знаком. После получения результата (4-х двойных слов), 1-е произведение складывается со 2-м, а 3-е с 4-м.

Действие команды, написанной в следующем формате показано на рисунке 1.9:

Pmaddwd MMX-регистр, операнд2

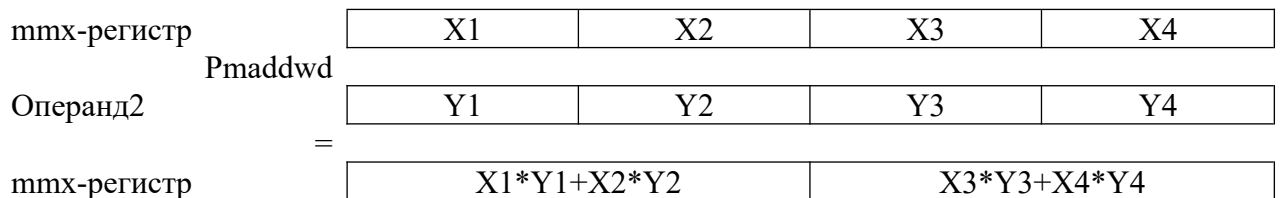


Рисунок 1.9 – Действие команды pmaddwd.

Pmulhw MMX-регистр, операнд2

Попарное умножение слов, в результате получают двойные слова, но в выходном операнде (MMX-регистре) сохраняются старшие слова результатов.

Pmullw MMX-регистр, операнд2

Попарное умножение слов, в результате получают двойные слова, но в выходном операнде (MMX-регистре) сохраняются младшие слова результатов.

Алгоритм получения полного значения произведения:

1. Получить старшие слова произведений командой pmulhw.
2. Получить младшие слова произведений командой pmullw.
3. Объединить результаты командами punpckhwd и punpcklwd.

Следующие команды приведем без форматов и прочих подробностей.

Команды сравнения

Команды сравнения на равенство: pcmpqfb, pcmpqfw, pcmpeqd.

Команды сравнения на условие «больше»: pcmpgtb, pcmpgtw, pcmpgtd.

Дополнительные команды

Вычисление среднего: pavgb, pavgw

Вычисление максимума: pmaxub, pmaxsw

Вычисление минимума: pminub, pminsw

Перенос по маске: pmovmskb

3. ЗАДАНИЕ

Даны массивы знаковых чисел А, В, С. Вычислить массив Y согласно выражению по варианту.

Таблица 1.10 – Варианты заданий

1) A – байт, B – байт, C – слово. $Y = A \cdot B + C$	10) A – байт, B – байт, C – слово. $Y = A^2 - B \cdot C$
2) A – байт, B – слово, C – слово. $Y = A \cdot B + C$	11) A – слово, B – слово, C – байт. $Y = A^2 - B \cdot C$
3) A – слово, B – байт, C – байт. $Y = A \cdot B + C$	12) A – байт, B – байт, C – байт. $Y = A^2 - B \cdot C$
4) A – байт, B – байт, C – слово. $Y = A - B \cdot C$	13) A – слово, B – слово, C – слово. $Y = A^2 - B \cdot C$
5) A – слово, B – слово, C – байт. $Y = A - B \cdot C$	14) A – байт, B – байт, C – слово. $Y = A^2 + B \cdot C$
6) A – слово, B – байт, C – байт. $Y = A - B \cdot C$	15) A – слово, B – слово, C – байт. $Y = A^2 + B \cdot C$
7) A – байт, B – байт, C – слово. $Y = A^2 - B + C$	16) A – байт, B – байт, C – байт. $Y = A^2 + B \cdot C$
8) A – слово, B – слово, C – байт. $Y = A^2 - B + C$	17) A – слово, B – слово, C – слово. $Y = A^2 + B \cdot C$
9) A – байт, B – слово, C – байт. $Y = A^2 - B + C$	

1.4 Векторные операции над дробными числами

1. ЦЕЛЬ РАБОТЫ

Познакомиться с системой команд SSE.

2. УКАЗАНИЯ К РАБОТЕ

SSE (Streaming SIMD Extensions) предназначена для векторных операций над числами с плавающей точкой: массивами из 4-х чисел формата R4 (float).

Регистры

В SSE используются 128-разрядные регистры xmm0, xmm1, ..., xmm7 и 32-х разрядный регистр управления/состояния **mxcsr**, который является аналогом регистров состояния (swr) и управления (swr) мат.сопроцессора (FPU).

В процессе обработки данных могут возникать исключительные ситуации, которые устанавливают 0-5 биты регистра **mxcsr**:

- 1) некорректная операция (invalid operation) – бит 0
- 2) денормализованный операнд – бит 1
- 3) деление на 0 – бит 2
- 4) арифметическое переполнение – бит 3
- 5) потеря значащих разрядов – бит 4
- 6) потеря точности – бит 5

Маскирование исключений устанавливается путем установки в 1 битов с 7 по 12 соответственно (при этом исключение будет обрабатываться стандартным алгоритмом).

Биты 13-14 аналог поля RC в FPU (задают режим округления).

Команды

суффиксы для арифметических команд:

ps – операция над массивом чисел (массивом пар чисел)

ss - операция над парой чисел (записанных в младших разрядах регистров)

суффиксы для команд копирования:

ap – выравненные данные (адрес операнда в памяти кратен 16-ти)

up – невыравненные данные (адрес операнда в памяти НЕкратен 16-ти)

s – операнды одинарной точности, т.е. R4.

Копирования данных

Команды этой группы выполняют операции параллельного копирования упакованных элементов данных, а также скалярного копирования только младшего элемента операнда. Действия производятся над операндами, расположенными в XMM-регистрах или в памяти.

Командами параллельного копирования являются: **movaps**, **movups**, **movhps**, **movhlps**, **movlhps**, **movlps**, **movmskps**, **movss**.

Команда **movaps** (Move aligned packed, single-precision, floating-point) копирует 128 бит упакованных данных из памяти в SIMD-регистр с плавающей точкой и наоборот, или между SIMD-регистрами с плавающей точкой. Линейный адрес соответствует адресу младшего байта группы данных в памяти. Обращение в память должно быть по адресу, кратному 16 байтам; в противном случае генерируется исключение.

Команда **movups** (Move unaligned packed, single-precision, floating-point) копирует 128 бит упакованных данных из памяти в SIMD-регистр с плавающей точкой и наоборот, или между SIMD-регистрами с плавающей точкой. Команда **MOVUPS** применяется для обращения к *невыровненным* данным в памяти.

Если данные выровнены, то лучше пользоваться командой **movaps**, так как она работает быстрее

Команда **movhps** (Move unaligned, high packed, single-precision, floating-point) копирует 64 бита упакованных данных из памяти в два старшие элемента SIMD-регистра с плавающей точкой и наоборот. При копировании данных из памяти в SIMD-регистр, два младших элемента этого регистра не изменяются. Команда работает с адресом младшего байта группы данных в памяти.

Команда **movlps** (Move unaligned, low packed, single-precision, floating-point) копирует 64 бита упакованных данных из памяти в два младшие элемента SIMD-регистра с плавающей точкой и наоборот. При копировании данных из памяти в SIMD-регистр, два старших элемента этого регистра не изменяются. Линейный адрес соответствует адресу младшего байта группы данных в памяти.

Команда **movhlps** (Move high to low packed, single-precision, floating-point) копирует старшие 64 бита из исходного регистра в младшие 64 бита регистра назначения. Старшие 64 бита регистра назначения не изменяются. Оба операнда должны быть регистрами!

Команда **movlhps** (Move low to high packed, single-precision, floating-point) копирует младшие 64 бита из исходного регистра в старшие 64 бита регистра назначения. Младшие 64 бита регистра назначения не изменяются. Оба операнда должны быть регистрами!

Команда **movmskps** (Move mask packed, single-precision, floating-point) копирует содержимое старших (знаковых) разрядов каждого из четырех FP-элементов, находящихся во входном регистре, в младшие разряды выходного целочисленного регистра, и формирует таким образом 4-битовую маску. Старшие 28 разрядов целочисленного регистра обнуляются.

Команда **movss** (Move scalar single-precision, floating-point) копирует младшие 32 бита из памяти в SIMD-регистр с плавающей точкой или наоборот, и между регистрами. При копировании 32 битов данных из памяти в SIMD-регистр, все 96 оставшихся старших битов этого регистра обнуляются. Команда работает с адресом младшего байта группы данных в памяти.

Арифметические команды

Арифметические команды SIMD выполняют сложение, вычитание, умножение и деление данных одинарной точности с плавающей точкой.

Входной (второй) операнд этих команд может располагаться либо в SIMD-регистре, либо в памяти. Выходной (первый) операнд должен находиться в SIMD-регистре. Арифметические команды поддерживают как параллельные, так и скалярные операции.

Команды **addps** (Add packed, single-precision, floating-point) и **subps** (Subtract packed, single-precision, floating-point) складывают или вычитают четыре пары упакованных операндов одинарной точности с плавающей точкой.

Команды **addss** (Add scalar single-precision, floating-point) и **subss** (Subtract scalar single-precision, floating-point) складывают или вычитают младшую пару упакованных операндов одинарной точности с плавающей точкой. Остальные элементы регистра не меняются.

Команды **mulps** (Multiply packed, single-precision, floating-point) и **divps** (Divide packed, single-precision, floating-point) умножают или делят четыре пары упакованных операндов одинарной точности с плавающей точкой.

Команды **mulss** (Multiply scalar single-precision, floating-point) и **divss** (Divide scalar single-precision, floating-point) умножают или делят младшую пару упакованных операндов одинарной точности с плавающей точкой. Остальные элементы регистра не меняются.

Команды вычисления квадратных корней **sqrtps** (Square root packed, single-precision, floating-point) и **sqrtss** (Square root scalar single-precision, floating-point) определяют арифметические значения квадратного корня для чисел, упакованных во входной (второй) операнд, и записывают результаты в элементы выходного операнда.

Входной операнд для этих команд может располагаться либо в регистре, либо в памяти. Выходной операнд должен обязательно находиться в SIMD-регистре.

Команды вычисления квадратных корней поддерживают как параллельные, так и скалярные операции (**sqrtps** и **sqrtss**).

Команды нахождения минимума и максимума производят попарное сравнение соответствующих элементов двух операндов и записывают минимальные или максимальные значения в выходной операнд.

Входной операнд для этих команд может располагаться либо в XMM-регистре, либо в памяти. Выходной операнд должен обязательно находиться в XMM-регистре. Поддерживаются как параллельные, так и скалярные операции (**maxps** (Maximum packed, single-precision, floating-point), **maxss**, **minps** и **minss**).

Команды упаковки/распаковки

unpckhps – параллельное перемещение старших двойных слов из операнда-источника и операнда-приемника в операнд приемник (см. таблицу 1.11).

Таблица 1.11 – Действие команды **unpckhps**

Приемник (xmm-рег)				Источник (xmm-рег или 128-бит.яч.)			
п3	п2	п1	п0	и3	и2	и1	и0
и3	п3	и2	п2	и3	и2	и1	и0

unpcklps – параллельное перемещение младших двойных слов из операнда-источника и операнда-приемника в операнд приемник (см. таблицу 1.12).

Таблица 1.12 – Действие команды unpcklps

Приемник (xmm-рег)				Источник (xmm-рег или 128-бит.яч.)			
п3	п2	п1	п0	и3	и2	и1	и0
И1	П1	И0	П0	и3	и2	и1	и0

shufps <приемник>, <источник>, <маска 8бит>

Старшие 4 бита маски показывают номера чисел источника (представленные парами бит), которые становятся старшими упакованными числами результата (приемника). Младшие 4 бита маски показывают номера чисел приемника (представленные парами бит), которые становятся младшими упакованными числами результата (приемника). Пример действия команды со значением маски 0x11 (шестнадцатеричное значение) показано в таблице 1.13.

Таблица 1.13 – Пример действия команды shufps со значением маски 0x11

Приемник (xmm-рег)				Источник (xmm-рег или 128-бит.яч.)			
п3	п2	п1	п0	и3	и2	и1	и0
И1	П1	И0	П0	и3	и2	и1	и0

3. ЗАДАНИЕ

Даны массивы из n дробных чисел (float): A, B, C, D, E. Вычислить математическое выражение заданное вариантом, используя команды SSE (см. таблицу 1.2).

2. Список рекомендуемой литературы

2.1. Основная литература

1. Пятибратов, Александр Петрович. Вычислительные системы, сети и телекоммуникации: учебник для вузов / А. П. Пятибратов, Л. П. Гудыно, А. А. Кириченко; ред. А. П. Пятибратов. - 3-е изд., перераб. и доп. - М.: Финансы и статистика, 2006. - 558 с. (90 экз. библиотека ТУСУР)
2. Павловская, Татьяна Александровна. С/С++. Программирование на языке высокого уровня: учебник для вузов / Т. А. Павловская. - СПб.: Питер, 2007. - 464 с. (47 экз. библиотека ТУСУР).
3. Магда Ю.С. Ассемблер для процессоров Intel Pentium. – СПб.: Питер, 2006. – 410 с.

2.2. Дополнительная литература

1. Юров, Виктор Иванович. Assembler: Учебное пособие для вузов / В. И. Юров. - 2-е изд. - СПб.: Питер, 2005. - 636с. (20 экз. библиотека ТУСУР)
2. Абель, Питер. Язык Ассемблера для IBM PC и программирования: Пер. с англ. / П. Абель; пер. Ю. В. Сальников. - М.: Высшая школа, 1992. - 447 с. (24 экз. библиотека ТУСУР)

2.3. Перечень пособий, методических указаний и материалов, используемых в учебном процессе

1. Фефелов, Николай Петрович. Организация ЭВМ и систем. Введение в ассемблер: учебное пособие к лабораторным работам для студентов специальности 230105 - Программное обеспечение вычислительной техники и автоматизированных систем / Н. П. Фефелов; Федеральное агентство по образованию, Томский государственный университет систем управления и радиоэлектроники, Кафедра автоматизированных систем управления. - Томск: ТУСУР, 2006. - 51 с. (85 экз. библиотека ТУСУР)